

Cellpose Segmentation Plan

2026.07.21 - Decision and plan: move colony segmentation to Cellpose

Related: [[experiments.010-kuzmin-tmi.12_panel_crispr_fitness_assay]] (the assay + the classical `torchcell/sga/image.py` pipeline this replaces), `torchcell/sga/image.py` (current segmenter).

Decision

Adopt **Cellpose** (Cellpose-SAM / cyto3) for per-colony segmentation, replacing the classical per-cell MAD/border threshold in `torchcell/sga/image.py`. Rationale, after benchmarking every classical option (threshold, convex hull, Chan-Vese, morphological active contour) on our own colonies:

- Every classical *smoothing* method hallucinates on empty/tiny cells (active contour balloons to fill an empty well; hull deletes small colonies), so the threshold could only ever be a presence-gate, not a clean boundary. Classical is at its ceiling for this imaging.
- Cellpose returns **instance masks** – each colony gets its own integer ID – so overlapping / touching / off-grid extra colonies are separated natively. This directly solves the “invalidate a well that has a second, off-grid colony” requirement (two instances mapping to one grid node -> invalidate) and the merged-double case that distance-transform peak-splitting could not catch (thick necks give one ridge, not two peaks).
- Morphology-independent, no per-plate threshold tuning; zero-shot capable (no fine-tuning needed to start). We already have GPU (gila workstation, MIG-partitioned), so inference cost is a non-issue.

Key realization that de-risks this

At **full resolution the colonies are clean, sharp discs with no shadow tails** (see the crop experiments/019-echo-crispr-... Much of the edge raggedness and “shadow tails” we fought was an artifact of the **low-res 1499x1999 preview images being downscaled to 1400 px** for processing, not the real imaging. So Cellpose (and even the classical threshold) will do materially better on the full-res source. Corollary imaging SOP: export full-res originals (never the dragged Photos preview), and consider a fully diffuse backlight to further flatten any residual directional shading.

Web-demo validation (2026.07.21) - it works

Uploaded the full-res crop to the cellpose.org demo (cyto3, zero-shot, no tuning): predicted outlines hug every colony tightly and each colony becomes its own instance mask – visibly cleaner than the classical threshold, and touching colonies are separated. This green-lights the plan.

Downloaded instance-mask PNG for reference: `assets/images/019-echo-crispr-array/cellpose/P2_5nL_t72_crop_center...`

Compute + data move (to gila workstation)

- Work moves to the **gila workstation** (GPU). Transfer the run-2 images to the analogous location under the data root there, e.g. `/home/michaelvolk/Documents/projects/torchcell/torchcell-scratch`.
- rsync (from this M1 machine; adjust host alias as configured):

```
rsync -avP \
  experiments/019-echo-crispr-array/data/run2_2026-07-17/ \
  gila:/home/michaelvolk/Documents/projects/torchcell/torchcell-scratch/019-echo-crispr-array/run2_2026-07-17/
```

Only the **full-res *_up.jpg** captures should transfer (the low-res previews and the mislabeled archive were deleted – see `notes/scratch.2026.07.21.174048-del.md`). Verify sha256 after transfer (t72: `e2b7c59b... P1, c9118d17... P2`).

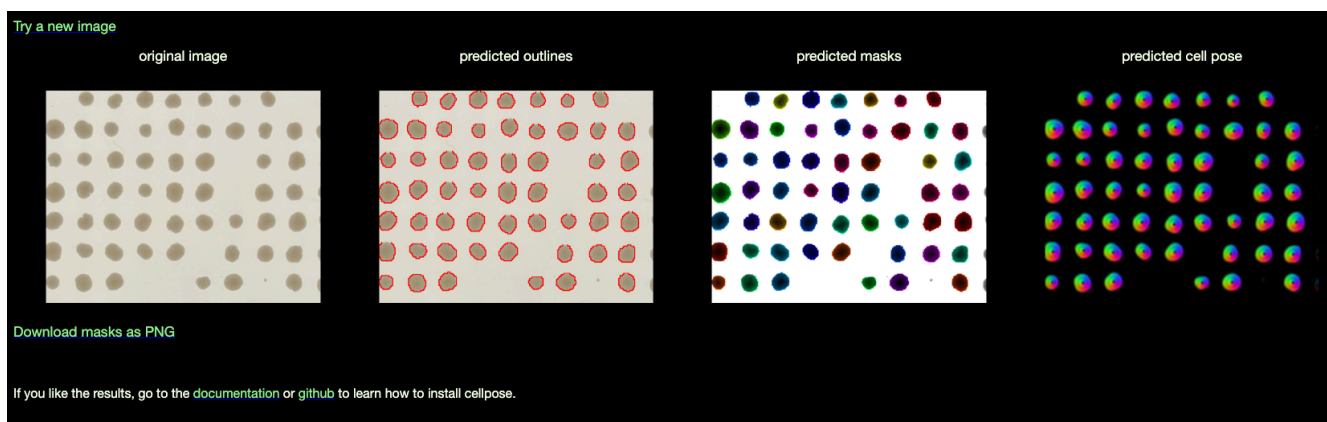


Figure 1: Cellpose cyto3 (zero-shot) on the full-res P2 5 nL 72 h crop: original, predicted outlines (tight to each colony), instance masks (one colour per colony), and cell-pose flow.

Cellpose integration plan

1. **Sanity check on the web demo (now):** upload the crop above to the Cellpose-SAM Hugging Face Space (or cellpose.org). Model `cyto3`; invert OFF (colonies are darker than agar); let it auto-estimate diameter first, then pin diameter $\sim$ the colony pixel size if needed. Confirm it instance-labels each disc and splits touching pairs.
2. **Local install on gila:** follow the official install + model-download instructions at <https://github.com/mouseland/cellpose> (`pip install cellpose[gui]` or `pip install cellpose`; pulls torch). Models auto-download on first use to `~/.cellpose/models`; `cyto3` is the default generalist. For an offline/pinned run, pre-fetch with `python -c "from cellpose import models; models.CellposeModel(gpu=True, model_type='cyto3')"` on a networked node, then sync `~/.cellpose/models`. (ONNX `cpsam` export `keekrej/cellpose-cpsam-onnx` is a torch-light alternative.) Verify GPU inference before wiring in.
3. **Wire into the pipeline (keep the grid):** the lattice fit in `image.py` (`_detect_blobs_backlit` $\rightarrow$ `_fit_lines` $\rightarrow$ `nodes`) is still the right well-assignment backbone and should be KEPT. New flow per plate:
 - Run Cellpose once on the whole processed plate (or per-cell crop) $\rightarrow$ instance masks.
 - Assign each instance to its nearest grid node (centroid within `~node_tol * pitch`).
 - `size` = instance mask pixel area; centroid = instance centroid.
 - **Invalidate** (M flag) any node that receives ≥ 2 instances, or an instance that is off every node $\rightarrow$ a stray/contaminant colony. This is the “multiple colonies not in the grid $\rightarrow$ invalidate” rule, now exact.
 - Keep the gel-polygon gate (`_gel_polygon`) to drop instances outside the gel.
 - Add a new `seg_method='cellpose'` branch (parallel to `'threshold'` / `'watershed'`), so the classical path stays available and comparable.
4. **Validation vs classical:** run both on all six run-2 plates; compare per-colony size correlation, WT CV (target: $\leq$ the classical ~ 0.11 - 0.17), Costanzo Pearson/Spearman (current best 2.5 nL 50 h: $r = 0.76$, $\rho = 0.69$), and the number of overlap/merged wells recovered vs invalidated. Only promote Cellpose to default if it holds or beats these.

Open questions

- Full-res vs downscaled input to Cellpose (full-res is cleaner but slower; `cyto3` has a diameter param that rescales internally – likely feed full-res per-plate crops).
- Dependency weight: `cellpose` + torch GPU in the `torchcell` env; keep it an optional extra so the classical path runs without it.
- Whether to fine-tune on a few hand-labeled plate crops if zero-shot `cyto3` under-segments the faint 2.5 nL colonies (the SegFormer organoid checkpoint `ReyaLabColumbia/Segformer_Organoid_Counter` is the closest analog if a semantic model is preferred, but it needs a separate split step).

Provenance

- Test crop: `experiments/019-echo-crispr-array/cellpose_test/P2_5nL_t72_crop_center.png` (946x730, full-res, from `P2_5nL_view_t72_up.jpg`), plus `..._crop_1024.png`.

- Reference tool studied: `sgatools` (Boone/Andrews SGA web app) added to the VS Code workspace; its image analysis calls `gitter` (public/gitter/gitter.R), the method the classical pipeline already follows.

2026.07.21 - GPU env set up + Cellpose module landed + validated on all six run-2 plates

Implemented the plan on the gila workstation (4x RTX 6000 Ada, slurm partition main, gpu:rtx6000:4). Cellpose is now a working, benchmarked alternative segmenter.

Environment (torchcell conda env, GilaHyper)

- `pip install cellpose -> cellpose 4.2.1.1` (Cellpose-SAM). NOTE the model is `cpsam`, not `cyto3`: cellpose 4.x replaced the v3 generalist lineage with a single SAM-based super-generalist. `models.CellposeModel(gpu=True)` loads it (diameter-agnostic; no diameter tuning). Weights auto-download to `~/.cellpose/models/cpsam_v2` (1.15 GB).
- **torch/torchvision ABI gotcha (important):** `pip install cellpose` pulled `torchvision` from PyPI, which mismatched the pinned `torch 2.11.0+cu128 -> RuntimeError: operator torchvision::nms does not exist on import`. Fix: reinstall the cu128-matched build `pip install --no-deps --force-reinstall --index-url https://download.pytorch.org/whl/cu128 torchvision==0.26.0`. Verify with `from torchvision.ops import nms`.
- `pip install scikit-image (0.26.0)` was also needed – `image.py`'s `_gel_polygon` imports `skimage.draw`; the classical path had only ever run on the M1 Mac, so it was absent here.
- GPU smoke test on the committed full-res crop: model on `cuda:0`, eval **0.8 s -> 55 instance masks**, areas ~3.1-3.7 k px. Full 384 plate at full-res: ~55 s/plate.

Code (branch paper/figures-fig1)

- `torchcell/sga/cellpose_seg.py` (new): `quantify_plate_image_cellpose(path, model, cfg, overlay_path=, return_masks=) + CellposeSegConfig(pydantic) + PlateSegResult + load_cellpose_model`. It REUSES `image.py`'s lattice helpers (`_detect_blobs_backlit -> _fit_lines -> _gel_polygon -> _signed_dist`) verbatim – the classical `image.py` is untouched – and returns the SAME `[row,col,size,circularity,flag]` schema so `normalize_plate/score_plate` consume it unchanged. Well-assignment rules are now exact: an instance within `node_tol*pitch` of a node IS that well; a second instance on the node (or a near-stray within `stray_tol*pitch`) -> flag M; an instance off every node -> counted off-grid contaminant; instances outside the gel hexagon are dropped. Exported from `torchcell/sga/_init_.py`; `cellpose` added to `mypy` untyped-imports; `cellpose_seg` added to the `pydantic disallow_subclassing_any=false` override. `ruff+format+mypy` clean.
- `experiments/019-echo-crispr-array/scripts/run2_cellpose_segmentation.py` (new): imports the run-2 flow (`run2_volume_timepoints` conditions, orientation resolver, geometry) verbatim and swaps ONLY the segmenter. Crops each plate at FULL resolution (no 1400-px downscale – that downscale was the source of the shadow tails) and runs both Cellpose and classical on the SAME pixels for an apples-to-apples comparison. Writes `run2_cellpose_*` CSVs to `results/` and QC overlays to `assets/images/019-echo-crispr-array/cellpose/`.
- `experiments/019-echo-crispr-array/scripts/gh_cellpose_segmentation.slurm` (new): GilaHyper GPU job, `-p main --gres=gpu:rtx6000:1`, ~2 h wall. Do NOT hardcode `CUDA_VISIBLE_DEVICES` (slurm sets it from the gres alloc).
- Data moved M1 -> gila via `rsync` (30 files) to `/home/michaelvolk/Documents/projects/torchcell/torchcell-scratch/`; the worktree already holds the committed copies used for the validation run.

Validation (all 6 plate x timepoint images, Cellpose vs classical on identical full-res pixels)

`results/run2_cellpose_vs_classical.csv`. WT CV = colony-size CV across the on-plate BY4741 wells (lower = tighter reference = better assay); target was $\leq$ classical ~0.11-0.17.

group	occ cp/cl	M cp/cl	size r (P/S)	WT CV cp / cl
P1_t44	276 / 284	1 / 16	0.40 / 0.44	0.147 / 0.239
P2_t44	333 / 363	3 / 2	0.65 / 0.73	0.126 / 0.138
P1_t50	277 / 297	1 / 3	0.75 / 0.78	0.133 / 0.130

group	occ cp/cl	M cp/cl	size r (P/S)	WT CV cp / cl
P2_t50	343 / 363	3 / 0	0.85 / 0.86	0.126 / 0.123
P1_t72	285 / 297	1 / 2	0.67 / 0.68	0.138 / 0.167
P2_t72	319 / 353	35 / 16	0.45 / 0.48	0.180 / 0.349

Reading it:

- **WT CV: Cellpose matches or beats classical on every plate, and wins big on the hard ones** – P1_t44 (0.147 vs 0.239) and the crowded/overgrown P2_t72 (0.180 vs 0.349, where all six blanks are occupied and classical's reference falls apart). Meets the promotion criterion.
- **Multi-colony M:** on the overgrown P2_t72, Cellpose separates and rejects **35** competing- colony wells vs classical's 16 – the instance masks catch merged pairs that thresholding fused into one oversized (fitness-corrupting) colony. Conversely at P1_t44 classical raised 16 M flags to Cellpose's 1: those look like classical splitting one colony into blob+speck, i.e. false Ms Cellpose avoids.
- **off-grid contaminants = 0** on all plates; every instance landed on the array.
- Cellpose is slightly more conservative on occupancy (rejects faint/edge wells the threshold keeps) – expected, and desirable for a fitness readout.
- **Circularity saturates at ~1.0** under Cellpose (masks are smooth by construction), so the C low-circularity QC flag loses discriminative power – lean on WT CV + size agreement instead.
- Visual QC overlays (`run2_cellpose_overlay_<group>.png`) confirm tight per-colony boundaries, faint/empty wells correctly unmarked, and no frame/shelf/reflection detections.

Next

- Promote `seg_method='cellpose'` to the run-2 default once the above holds on a re-image with the diffuse-backlight SOP; keep classical available for comparison.
- Consider a light fine-tune only if the faint 2.5 nL (P1) colonies under-segment; zero-shot `cpsam` already handled them here.